

Logs, more sorting, inheritance, and Set

4/25/25

Questions about lab?

- Searching for treasure using *depth-first search*
 - Go in one direction as much as possible and then backtrack to try a different direction

Cartographers

- Game where you draw pieces of terrain onto a map to meet various scoring goals
 - Next HW is going to be implementing some of the scoring criteria
- Real game is in the Math/CS lounge for you to play

Recall: Logarithms

$\log_b x$ = “log base b of x”

= power of b that gives x

= number of times you can divide x by b
before getting 1

$2^0 = 1$ so $\log_2 1$ is 0

$2^1 = 2$ so $\log_2 2$ is 1

$2^2 = 4$ so $\log_2 4$ is 2 (and $\log_2 3$ is between 1 & 2)

Where we left off: What is $\log_2 32$?

A. 3

B. 4

C. 5

D. 6

E. None of the above

What is $\log_2 100$?

- A. Between 6 and 7
- B. Between 7 and 8
- C. Between 8 and 9
- D. Between 9 and 10
- E. None of the above

What is $\log_2 100$?

- A. Between 6 and 7
- B. Between 7 and 8
- C. Between 8 and 9
- D. Between 9 and 10
- E. None of the above

What is $\log_2 1000$?

- A. Between 7 and 8
- B. Between 8 and 9
- C. Between 9 and 10
- D. Between 10 and 11
- E. None of the above

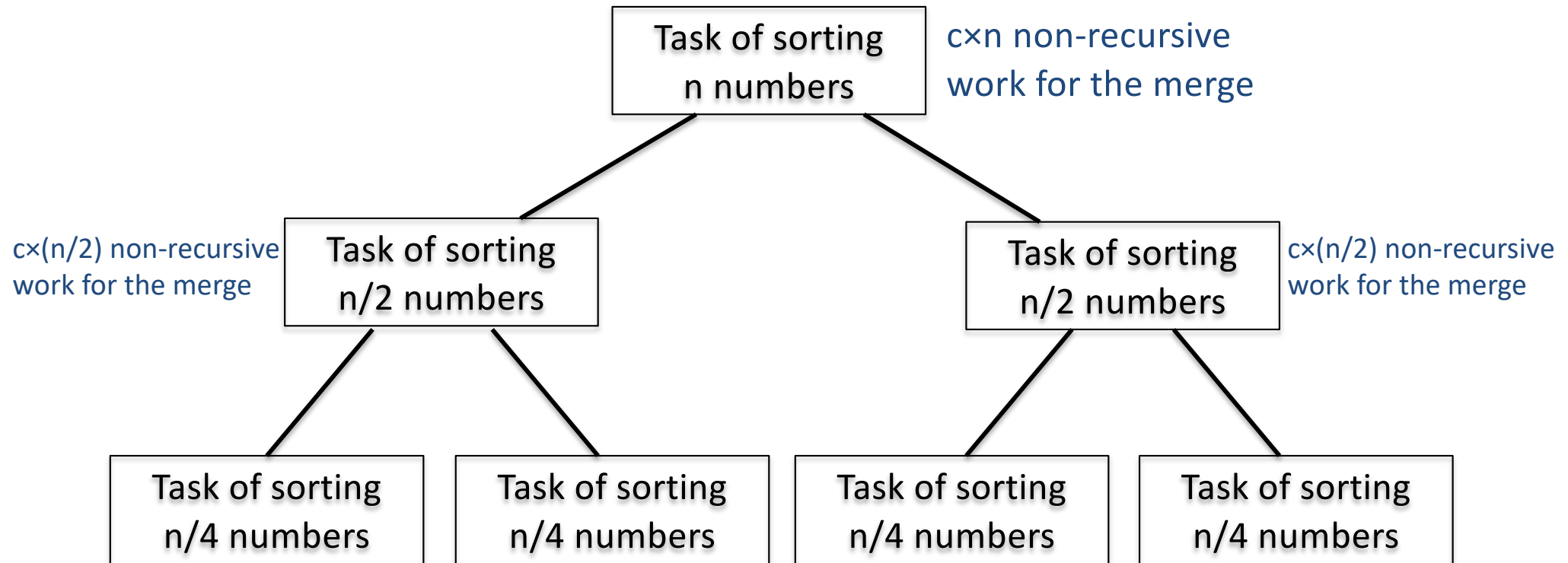
What is $\log_2 1000$?

- A. Between 7 and 8
- B. Between 8 and 9
- C. Between 9 and 10
- D. Between 10 and 11
- E. None of the above

Properties of logarithms

- $\log_b xy = \log_b x + \log_b y$
- $\log_b (x/y) = \log_b x - \log_b y$
- $\log_b x^y = y \log_b x$
- $\log_b a = \frac{\log_c a}{\log_c b}$ for any c

Recall: Running time of mergesort



... and so on

Quicksort

```
void quicksort(int[] a, int lo, int hi) { //sort a[lo..hi]
    if(hi <= lo) return;    //base case: empty array

    //split a into values <= pivot and >= pivot
    int j = partition(a, lo, hi);    //j = index of pivot

    quicksort(a, lo, j-1);           //sort left part
    quicksort(a, j+1, hi);           //sort right part
}
```

Partition

```
int partition(int[] a, int lo, int hi) {  
    int i = lo, j = hi+1;           //left and right scan indices  
    int x = a[lo];                 //pivot value  
    while(true) {  
        while(a[++i] < x) if(i == hi) break;  
        while(x < a[--j]) if(j == lo) break;  
        if(i >= j) break;  
        swap(a, i, j);             //swap a[i] & a[j]  
    }  
    swap(a, lo, j);  
    return j;  
}
```

Partition

```
int partition(int[] a, int lo, int hi) {  
    int i = lo, j = hi+1;           //left and right scan indices  
    int x = a[lo];                 //pivot value  
    while(true) {  
        while(a[++i] < x) if(i == hi) break;  
        while(x < a[--j]) if(j == lo) break;  
        if(i >= j) break;  
        swap(a, i, j);             //swap a[i] & a[j]  
    }  
    swap(a, lo, j);  
    return j;  
}
```

What is partition's running time?

A) $O(1)$

D) $O(n^2)$

B) $O(\log n)$

E) None of the

C) $O(n)$

above

Partition

```
int partition(int[] a, int lo, int hi) {  
    int i = lo, j = hi+1;           //left and right scan indices  
    int x = a[lo];                 //pivot value  
    while(true) {  
        while(a[++i] < x) if(i == hi) break;  
        while(x < a[--j]) if(j == lo) break;  
        if(i >= j) break;  
        swap(a, i, j);             //swap a[i] & a[j]  
    }  
    swap(a, lo, j);  
    return j;  
}
```

What is partition's running time?

- | | |
|-----------------------------|----------------|
| A) $O(1)$ | D) $O(n^2)$ |
| B) $O(\log n)$ | E) None of the |
| C) <u>$O(n)$</u> | above |

Running time of Quicksort